# Python Engineering - Semana 01

Semana 01 - Lógica y fundamentos

```
01_python_bootcamp/week_01_logic/
```

## Cómo usar este documento

Este PDF es un índice de repaso de la semana. Cada class resume **qué problema resolvemos, qué conceptos introducimos y qué sintaxis o funciones aparecen**. Si un concepto se te olvida, localízalo aquí y vuelve al ejercicio original.

## class_01_equation_to_algorithm

**EX01 - Presión dinámica**

**Problema:** Convertir una ecuación de ingeniería en una secuencia de instrucciones Python.

**Conceptos:** problema -> entradas -> ecuación -> algoritmo -> salida; variables; asignación; expresiones numéricas.

**Tipos:** int = número entero; float = número real con decimales; str = texto; bool = True/False.

**Sintaxis:** x = value, operadores + - * / ** y print(...).

**Idea clave:** Una variable es un nombre que referencia un valor; una ecuación matemática debe convertirse en operaciones ejecutables.

## class_02_conditionals

**EX02 Ecuación del cohete | EX03 Ecuación cuadrática | EX04 Coeficiente de arrastre por Mach**

**Problema:** Hacer que el programa tome decisiones según el valor de los datos.

**Conceptos:** condiciones booleanas, comparaciones, ramas mutuamente excluyentes, casos especiales y dominio físico/matemático.

**Sintaxis:** if, elif, else; ==, !=, <, <=, >, >=; and / or / not.

**Aplicaciones:** Tsiolkovsky, discriminante de una ecuación de segundo grado y coeficiente de arrastre por regiones de Mach.

**Idea clave:** El flujo del programa puede cambiar dependiendo de una condición.

## class_03_loops

**EX05 Suma de cuadrados | EX06 Factorial | EX07 Taylor exp | EX08 Movimiento vertical + Mini Proyecto 01**

**Problema:** Repetir un algoritmo y acumular o evolucionar resultados.

**Conceptos:** iteración, contador, acumulador, número de iteraciones, convergencia y actualización de estado.

**Sintaxis:** for, range(...), variables acumuladoras y actualización dentro del bucle.

**Métodos numéricos:** serie de Taylor de exp(x) y método de Euler para movimiento vertical.

**Proyecto:** Ballistic Flight Simulator v1 con estudio de sensibilidad y convergencia respecto a dt.

**Idea clave:** Un bucle aplica el mismo bloque varias veces; en métodos numéricos cada iteración suele representar un paso del modelo.

## class_04_functions

**EX09-EX12 - Funciones de ingeniería y movimiento vertical**

**Problema:** Encapsular cálculos para reutilizarlos y reducir duplicación.

**Conceptos:** función, parámetros, argumentos, return, ámbito local, validación y múltiples salidas.

**Sintaxis:** def function(...):, return value y llamadas function(...).

**Patrones:** validate_inputs(), salidas tempranas y retorno de varios valores mediante una tuple.

**Aplicaciones:** presión dinámica, ecuación del cohete, Taylor exp y movimiento vertical reorganizados con funciones.

**Idea clave:** Una función debe tener una responsabilidad clara, entradas bien definidas y una salida útil.

# Índice rápido de conceptos

| Concepto | Dónde verlo |
|---|---|
| Problema: Convertir una ecuación de ingeniería en una secuencia de instrucciones Python. | class_01_equation_to_algorithm |
| Conceptos: problema -> entradas -> ecuación -> algoritmo -> salida; variables; asignación; expresiones numéricas. | class_01_equation_to_algorithm |
| Tipos: int = número entero; float = número real con decimales; str = texto; bool = True/False. | class_01_equation_to_algorithm |
| Sintaxis: x = value, operadores + - * / ** y print(...). | class_01_equation_to_algorithm |
| Idea clave: Una variable es un nombre que referencia un valor; una ecuación matemática debe convertirse en pasos ejecutables. | class_01_equation_to_algorithm |
| Problema: Hacer que el programa tome decisiones según el valor de los datos. | class_02_conditionals |
| Conceptos: condiciones booleanas, comparaciones, ramas mutuamente excluyentes, casos especiales y dominio físico/matemático. | class_02_conditionals |
| Sintaxis: if, elif, else; ==, !=, <, <=, >, >=; and / or / not. | class_02_conditionals |
| Aplicaciones: Tsiolkovsky, discriminante de una ecuación de segundo grado y coeficiente de arrastre por regiones de Mach. | class_02_conditionals |
| Idea clave: El flujo del programa puede cambiar dependiendo de una condición. | class_02_conditionals |
| Problema: Repetir un algoritmo y acumular o evolucionar resultados. | class_03_loops |
| Conceptos: iteración, contador, acumulador, número de iteraciones, convergencia y actualización de estado. | class_03_loops |
| Sintaxis: for, range(...), variables acumuladoras y actualización dentro del bucle. | class_03_loops |
| Métodos numéricos: serie de Taylor de exp(x) y método de Euler para movimiento vertical. | class_03_loops |
| Proyecto: Ballistic Flight Simulator v1 con estudio de sensibilidad y convergencia respecto a dt. | class_03_loops |
| Idea clave: Un bucle aplica el mismo bloque varias veces; en métodos numéricos cada iteración suele representar un paso del modelo. | class_03_loops |
| Problema: Encapsular cálculos para reutilizarlos y reducir duplicación. | class_04_functions |
| Conceptos: función, parámetros, argumentos, return, ámbito local, validación y múltiples salidas. | class_04_functions |
| Sintaxis: def function(...):, return value y llamadas function(...). | class_04_functions |

# Regla de repaso

Primero busca **el concepto**; después abre el PDF de esa clase; por último ejecuta o modifica el ejercicio. La combinación documento + código es el mecanismo de repaso del curso.